A Thorough Breakdown of EfficientDet for Object Detection

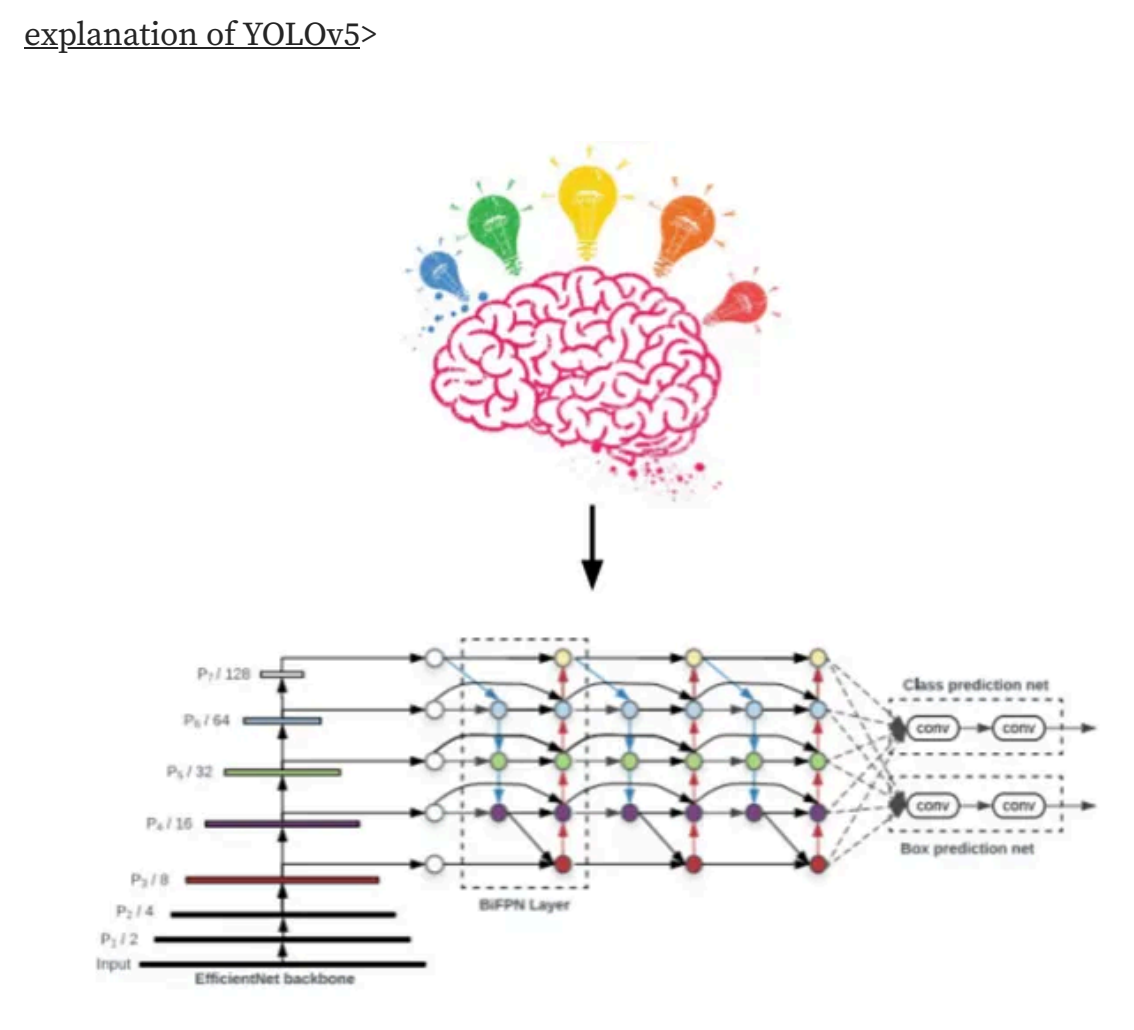
In this post, we do a deep dive into the structure of EfficientDet for object detection, focusing on the model's motivation, design, and architecture. We have also published A Thorough Breakdown of EfficientDet for Object Detection on our blog.

Jacob Solawetz · Follow · 8 min read · Apr 30, 2020

148 · 1

Recently, the Google Brain team published their EfficientDet model for object detection with the goal of crystallizing architecture decisions into a scalable framework that can be easily applied to other use cases in object detection. The paper concludes that **EfficientDet outperforms similar-sized models on benchmark datasets.** At Roboflow, we found that the base EfficientDet model generalizes to custom datasets hosted on our platform. (For an in-depth tutorial on implementing EfficientDet, please see this blog post on how to train EfficientDet and this Colab Notebook on how to train EfficientDet.)

In this blog post, we explore the rationale behind the decisions that were made in forming the final EfficientDet model, how EfficientDet works, and how EfficientDet compares to popular object detection models like YOLOv3, Faster R-CNN, and MobileNet. <and now an explanation of YOLOv4 and an explanation of YOLOv5>



Exploring the rationale inspiring EfficientDet's creation. Photo credit and citing the paper

> The founder of Mosaic Augmentation, Glen Jocher has released a new YOLO training framework titled YOLOv5. You may also want to see our post on YOLOv5 vs YOLOv4 This post will explain some of the pros of the new YOLOv5 framework, and help illuminate breakthroughs that have happened since the EfficientDet publication.
>
> YOLOv5 Breakdown

Challenges in Deep Learning EfficientDet Addresses

Before exploring the model, here are some key areas that have prevented image detection systems from being deployed to real life use cases.

1. **Data Collection** — With model architecture and pretrained checkpoints, EfficientDet cuts down on the amount of data required to generalize to a new domain.
2. **Model Design and Hyper Parameterization** — Once the data has been collected, machine learning engineers need to carefully set up the model design and tune a number of hyper parameters.
3. **Training Time** — the amount of time required to train the model on the gathered dataset. In the EfficientDet paper, this is measured in FLOPS (floating point operation per second).
4. **Memory Footprint** — Once the model is trained, how much memory is required to store the model weights when called upon for inference?
5. **Inference Time** — When the model is invoked, can it perform predictions quick enough to be used in a production setting?

Image Features and CNNs

The first step to training an object detection model is to translate the pixels of an image into features that can be fed through a neural network. Major progress has been made in the field of computer vision by using convolutional neural networks to create learnable features from an image. Convolutional neural networks mix and pool image features at different levels of granularity, allowing the model a choice of possible combinations to focus on when learning the image detection task at hand. However, the exact manner in which the convolutional neural network (ConvNet) creates features has been a keen area of interest in the research community for some time. ConvNet releases have included ResNet, NASNet, YoloV3, Inception, DenseNet, … and each has sought to increase image detection performance by scaling ConvNet model size and tweaking the ConvNet design. These ConvNet models are provided in a scaling fashion, so programmers can deploy a larger model to improve performance if their resources allow.

EfficientNet: Motivation and Design

Recently, the Google Brain team released their own ConvNet model called EfficientNet. **EfficientNet forms the backbone of the EfficientDet** architecture, so we will cover its design before continuing to the contributions of EfficientDet. EfficientNet set out to study the scaling process of ConvNet architectures. There are many ways — it turns out 🫧 - that you can add more parameters to a ConvNet.

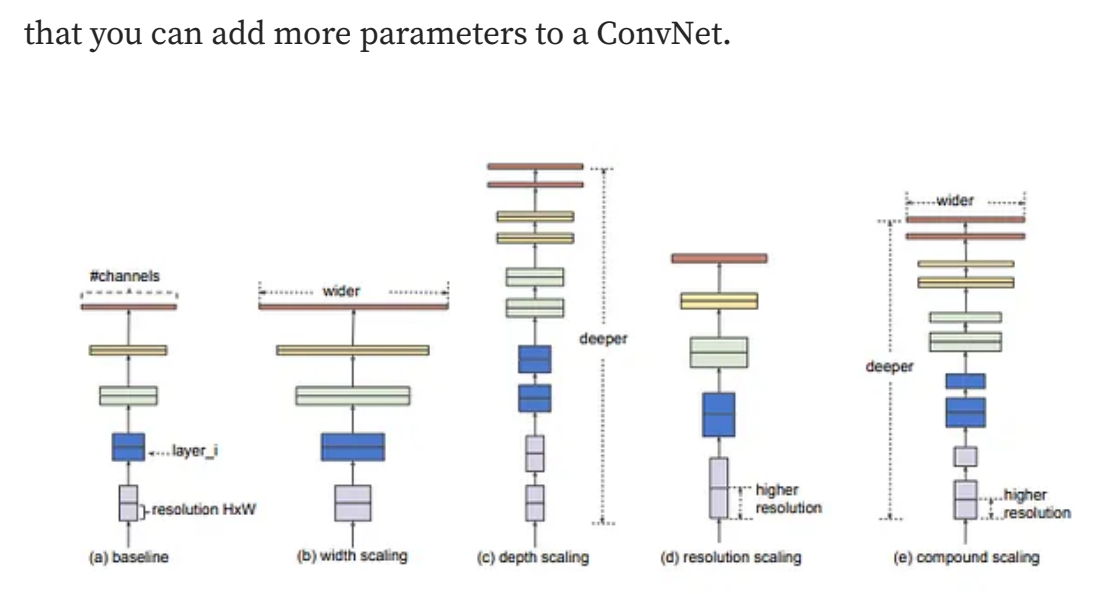


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Citing the paper

You can make each layer wider, you can make the number of layers deeper, you can input images at a higher resolution, or you can make a combination of these improvements. As you can imagine, the exploration of all these possibilities can be quite *tedious* for machine learning researchers. EfficientNet set out to define an automatic procedure for scaling ConvNet model architectures. The paper seeks to optimize downstream performance given free range over depth, width, and resolution while staying within the constraints of target memory and target FLOPs. They find that their scaling methodology improves the optimization of previous ConvNets as well as their EfficientNet architecture.

EfficientNet Network, Scaling and Evaluation

Creating a new model scaling technique is a big step forward, and the authors take their findings a step further by creating a new ConvNet architecture that pushes their state of the art results even higher. The new model architecture is discovered through neural architecture search. The neural architecture search optimizes for accuracy, given a certain number of FLOPS and results in the creation of a baseline ConvNet called EfficientNet-B0. Using the scaling search, EfficientNet-B0 is scaled up to EfficientNet-B1. The scaling function from EfficientNet-B0 to EfficientNet-B1 is saved and applied to subsequent scalings through EfficientNet-B7 because additional search becomes prohibitively expensive.

Blank pages welcome.
Fellowship. Encouragement. Prompts.
See you at The Writer's Circle.
Presented by Medium
Join now

The new family of EfficientNet networks is evaluated on the ImageNet leaderboard, which is an image classification task. Note: there have been some improvements since the original release of EfficientNet, including tweaks to optimize from the resolution discrepancy and deploying EfficientNet as a teacher and student.

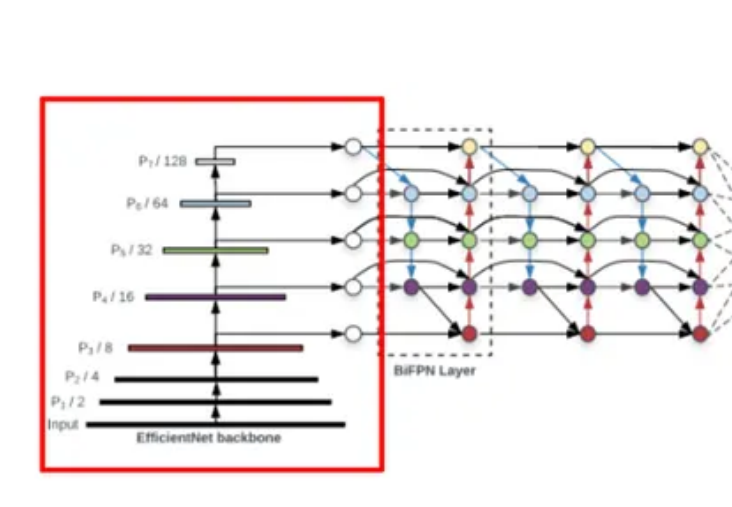
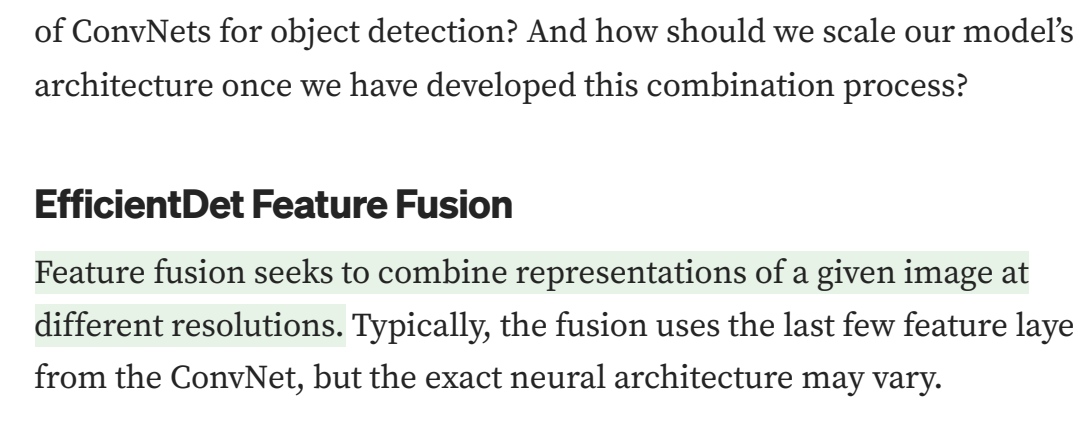


Figure 5. **FLOPS vs. ImageNet Accuracy** – Similar to Figure 1 except it compares FLOPS rather than model size.

Citing the paper

Pretty sweet! The EfficientNet looks like a good backbone to build upon. It efficiently scales with model size and outperforms other ConvNet backbones.

So far, we have covered the following portion of the EfficientDet network:



Citing the paper

Introducing EfficientDet

Now, we will move onto the contributions of EfficientDet, which seeks to answer the following question: how exactly should we combine the features of ConvNets for object detection? And how should we scale our model's architecture once we have developed this combination process?

EfficientDet Feature Fusion

Feature fusion seeks to combine representations of a given image at different resolutions. Typically, the fusion uses the last few feature layers from the ConvNet, but the exact neural architecture may vary.

Top highlight

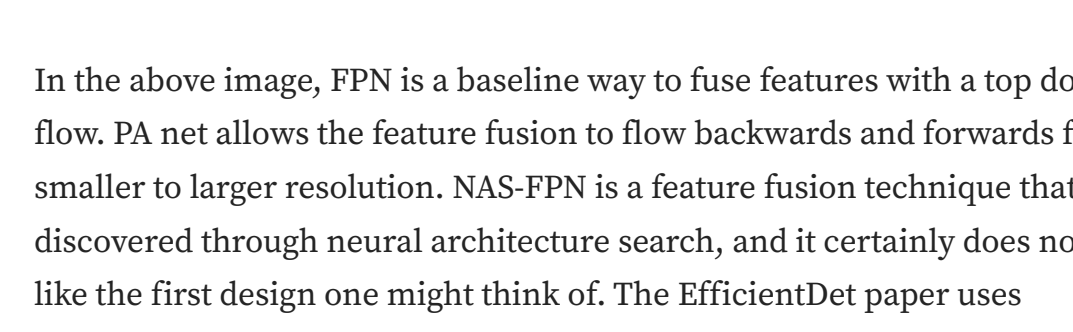


Figure 2: **Feature network design** – (a) FPN [23] introduces a top-down pathway to fuse multi-scale features from level 3 to 7 (P_3 - P_7); (b) PANet [26] adds an additional bottom-up pathway on top of FPN; (c) NAS-FPN [8] use neural architecture search to find an irregular feature network topology and then repeatedly apply the same block; (d) is our BiFPN with better accuracy and efficiency trade-offs.

Citing the paper

In the above image, FPN is a baseline way to fuse features with a top down flow. PA net allows the feature fusion to flow backwards and forwards from smaller to larger resolution. NAS-FPN is a feature fusion technique that was discovered through neural architecture search, and it certainly does not look like the first design one might think of. The EfficientDet paper uses "intuition" (and presumably many, many development sets) to edit the structure of NAS-FPN to settle on the BiFPN, a bidirectional feature pyramid network. The EfficientDet model stacks these BiFPN blocks on top of each other. The number of blocks varies in the model scaling procedure. Additionally, the authors hypothesize that certain features and feature channels might vary in the amount that they contribute to the end prediction, so they add a set of weights at the beginning of the channel that are learnable.

EfficientDet Model Scaling

Previous work on model scaling for image detection generally scaled portions of the network independently. For example, ResNet scales only the size of the backbone network. But a joint scaling function had not yet been explored. This approach is very reminiscent of the joint scaling work done to create EfficientNet.

The authors set up a scaling problem to vary the size of the backbone network, the BiFPN network, the class/box network, and the input resolution. The backbone network scales up directly with the pretrained checkpoints of EfficientNet-B0 through EfficientNet-B6. The BiFPN networks width and depth are varied along with the number of BiFPN stacks.

EfficientDet Model Evaluation and Discussion

The EfficientDet Model is evaluated on the COCO (Common Objects in Context) data set, which contains roughly 170 image classes and annotations across 100,000 images. COCO is considered to be the general purpose challenge for object detection. If the model performs well in this general domain, it will likely do very well on more specific tasks. EfficientDet outperforms previous object detection models under a number of constraints. Below, we look at the performance of the model as a function of FLOPS.

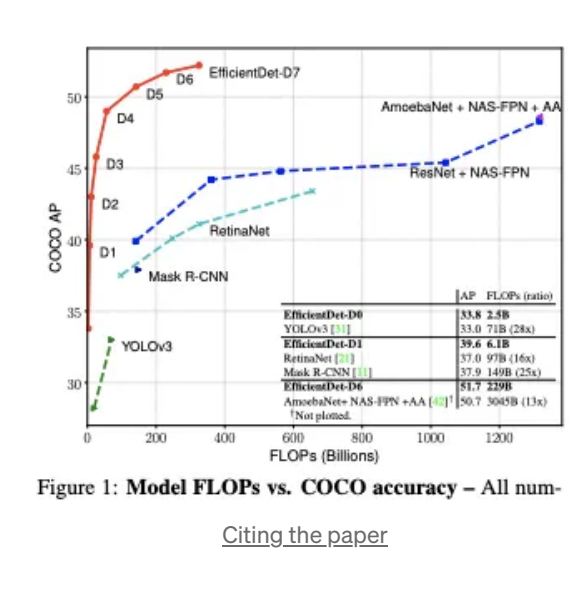


Figure 1: **Model FLOPs vs. COCO accuracy** – All num-

Citing the paper

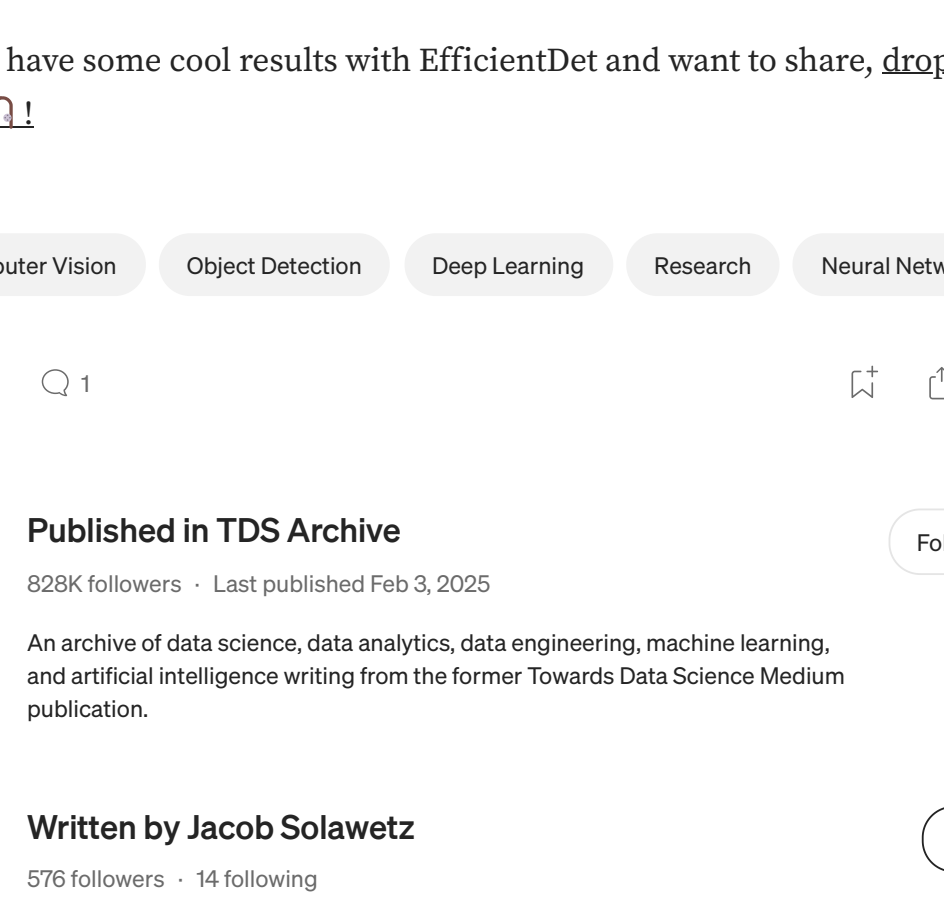
Here we can see that the model does quite well relative to other model families under similar constraints. The authors also evaluate the model on semantic segmentation on Pascal VOC. They find that they achieve state of the art there too. Hey, why not?

Simply, Why is EfficientDet Useful?

Taking a step back from the implementation details, it is pretty incredible to think about what the open sourced checkpoints of EfficientDet mean for computer vision engineers. **The pretrained checkpoints of EfficientNet crystallize all of the findings** and automaticity that the researchers at Google Brain placed into building a ConvNet, along with **all of the supervision that image classification on ImageNet can provide.** The EfficientDet checkpoints are **further leveraged with feature fusion and all components of the architecture are efficiently scaled.** Finally, these model weights are **pretrained on COCO**, a generalized image detection dataset. As a user, there are few decisions left up to question beyond the type of data to provide the model.

Nice Breakdown — How Do I Use EfficientDet?

At Roboflow, we have provided a tutorial on this blog post on how to train EfficientDet and this Colab Notebook on how to train EfficientDet. Through Roboflow, you can feed in your data set with annotations and simply feed a new data download link into our example and get some results. Then, after training, the notebook exports the trained weights for deployment to an application!



```
[26] from IPython.display import Image
     Image(filename='output.jpg')
```

Inference using efficientDet in our Colab Notebook

Roboflow is free to use, and you can host and transform up to 1000 images. Stay in Touch!

If you have some cool results with EfficientDet and want to share, drop us a line 🎧!

Computer Vision · Object Detection · Deep Learning · Research · Neural Networks

148 · 1

Published in TDS Archive
828K followers · Last published Feb 3, 2025
An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.
Following

Written by Jacob Solawetz
576 followers · 14 following
Author of WithAI, the world's premier AI publication providing practical advice for living in the new era, written on typewriter - https://withai.news
Follow

Responses (1)

Gabrielsferrante

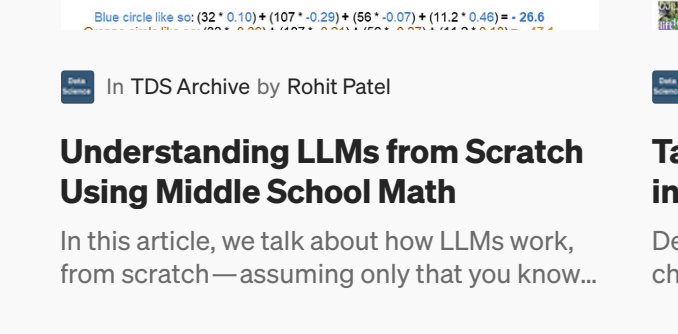
What are your thoughts?

Adel Saleh
May 19, 2020

Good job. Nice implementation, it works fine on colab but not in local machine.

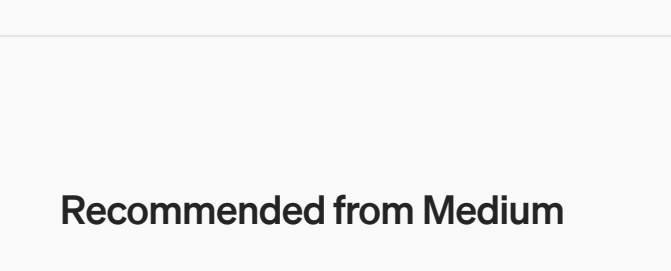
Reply

More from Jacob Solawetz and TDS Archive

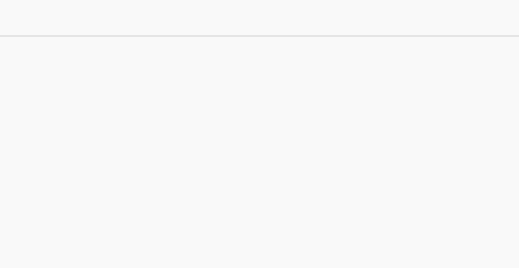


In TDS Archive by Jacob Solawetz
How to Train A Custom Object Detection Model with YOLO v5
In this post, we will walk through how you can train the new YOLO v5 model to recognize...
Jun 15, 2020 · 772 · 11

In TDS Archive by Steve Hedden
How to Implement Graph RAG Using Knowledge Graphs and...
A Step-by-Step Tutorial on Implementing Retrieval-Augmented Generation (RAG),...
Sep 6, 2024 · 2K · 20



In TDS Archive by Rohit Patel
Understanding LLMs from Scratch Using Middle School Math
In this article, we talk about how LLMs work, from scratch — assuming only that you know...
Oct 19, 2024 · 8.3K · 106



In TDS Archive by Jacob Solawetz
Tackling the Small Object Problem in Object Detection
Detecting small objects is one of the most challenging and important problems in...
Aug 25, 2020 · 257 · 2

See all from Jacob Solawetz · See all from TDS Archive

Recommended from Medium

In Towards AI by Ampatishan Sivalingam
The Death of CNNs: How Vision Transformers Rewrote Computer...
From AlexNet's 2012 revolution to ResNet's dominance, and why it all became obsolete...
Feb 5 · 248 · 9

In Generative AI by Adham Khaled
Stanford Just Killed Prompt Engineering With 8 Words (And I...
ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...
Oct 19, 2025 · 25K · 686

In Write A Catalyst by Dr. Patricia Schmidt
As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...
Most people do #1 within 10 minutes of waking (and it sabotages your entire day)
Jan 14 · 42K · 807

In Level Up Coding by Kusireddy
I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...
91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...
Dec 28, 2025 · 12K · 448

In Artificial Corner by The PyCoach
The Best AI Tools for 2026
If you're going to learn a new AI tool, make sure it's one of these
Dec 1, 2025 · 5.4K · 317

Kiran vutukuri
58. How AI Learns to Compare Everything:Triplet Loss and Metri...
Face recognition systems don't predict faces. They learn to measure. Given two faces, the...
Dec 20, 2025 · 57

See more recommendations